



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Follow the Name

CMSC 124 Activity 9

Prepared by: Rene Andre Bedonia Jocsing

Published: September 24, 2026

Deadline: September 25, 2026

When three active blocks all declare `x`, the rule use the nearest one isn't complete. Nearest in the source and nearest in the active call chain can be different places. Today you'll build both lookup paths before allowing the program to print anything.

The programs here are pseudocode. They're written to be read. Every rule they depend on is on this page. Laboratory Activity 4 is where these paths become code, since the chained environments that activity asks for implement the lookup paths you build today.

You have 60 minutes for the three checkpoints. Keep each worked example beside the checkpoint it supports. Discussion with classmates and the instructor is allowed and encouraged throughout. This activity is worth 16 points. Nine points reward scope constructions, and seven reward explanations.

A declaration's **scope** is the region where its identifier may be used. Its **lifetime** is the time when its storage exists. **Shadowing** happens when a nearer declaration hides another declaration of the same identifier.

Under **static scope**, also called lexical scope, lookup follows textual nesting: current block, enclosing block, then the next enclosing block. A **textual parent** is the surrounding block where a procedure is defined. Under **dynamic scope**, lookup follows active calls: current call, caller, caller's caller, and so on. An **active caller** is the procedure whose unfinished call invoked the current one.

A **lookup path** lists the scopes that one rule can check, in order. Lookup stops at the first matching declaration. A **referencing environment** contains the variable bindings visible at one program point after lookup resolves shadowing. Shadowed variables and procedure declarations don't enter that environment.

Worked Example for Checkpoint 1

Suggested time: 32 minutes, including the worked example.

Indentation shows textual nesting:

```
1  rate = 10
2
3  scope():
4      rate = 20
5
6      show():
7          print(rate)
8
9      invoke():
10         rate = 30
11         show()
12
```

```
13     invoke()  
14  
15     scope()
```

At `print`, all three `rate` cells are alive. Static lookup checks `show`, then its textual parent `scope`, where it finds 20. Dynamic lookup checks the active call `show`, then its caller `invoke`, where it finds 30. The two rules therefore print different values even though the active runtime state is the same.

Write the complete lookup path before selecting a variable. Then stop at the first matching `rate` and place only that binding in the referencing environment.

Rule	Complete lookup path for <code>rate</code>	First match	Referencing environment at <code>print</code>
static	<code>show -> scope -> global</code>	<code>scope.rate = 20</code>	<code>scope.rate = 20</code>
dynamic	<code>show -> invoke -> scope -> global</code>	<code>invoke.rate = 30</code>	<code>invoke.rate = 30</code>

The path records where lookup could proceed. The environment records what the statement can use. Global `rate` is shadowed under both rules, so it appears on each complete path but in neither environment.

The governing question is which path determines proximity. Choosing 30 for static lookup because `invoke` made the call follows the dynamic path under the wrong label. Sibling procedures don't become textual parents when one calls the other.

The two figures below are the same program point under the two rules. Only the lookup path changes.

The same reference follows two lookup paths

Static scope follows definitions in the source. Dynamic scope follows active callers.

Follow the static lookup path

`show` is defined inside `scope`, so lookup finds `scope.rate = 20`. The sibling `invoke` is not a textual parent.

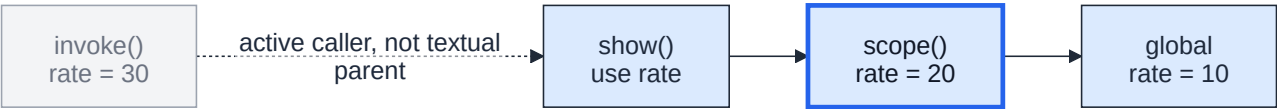


Figure 1: Follow the static lookup path

Follow the dynamic lookup path

`invoke` called `show`, so its live `rate = 30` is the first matching declaration. `scope` and the `global` block remain later on the path.

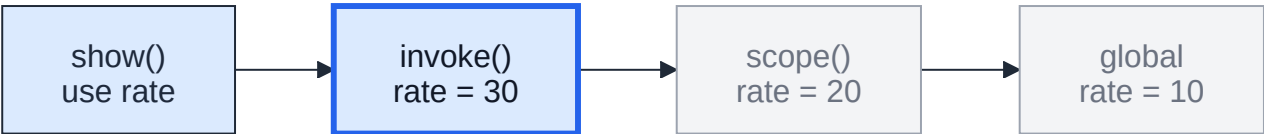


Figure 2: Follow the dynamic lookup path

Checkpoint 1: Build Both Paths (8 Points)**Running Case**

Use this pseudocode. Indentation shows which procedures are textually nested.

```

1  x = 1
2
3  outer():
4      x = 2
5
6      target():
7          y = 4
8          print(x + y)
9
10     caller():
11         x = 3
12         target()
13
14     caller()
15
16 outer()

```

Build each lookup path before selecting a declaration. Then record only the selected variables in the referencing environment. Two different things can stop a live declaration from being chosen.

A declaration can be alive but hidden. Global `x = 1` is alive the whole time, but under static scope `outer's x = 2` sits nearer and hides it.

A declaration can be alive but off the path entirely. `caller's x = 3` is alive at `print`, yet the static path out of `target` never passes through `caller`, so static lookup never reaches it.

At the `print` call, all three assignments to `x` have executed and their storage still exists.

1. (Construction: 5 points) Write the complete static lookup paths for `x` and `y`, and mark each first match. Record the selected bindings in the static referencing environment, then compute the printed value. Repeat these steps under dynamic scope. `y` is declared inside `target`, so both rules find it in the first scope they check.

2. (Explanation: 3 points) Answer both parts.

- Explain why the two results differ. Your answer must distinguish `target's` textual parent from its active caller.
- Under each rule in turn, say which `x` gets selected and which ones it shadows.

Scope Isn't Lifetime

During `target`, the local `x` belonging to `caller` is alive because `caller` hasn't returned. Static scope still doesn't let `target` access that cell because `caller` isn't a textual ancestor of `target`.

Worked Example for Checkpoint 2

Suggested time: 12 minutes, including the worked example.

```

1  rate = 10
2
3  scope():
4      rate = 20
5
6      show():

```

```

7         print(rate)
8
9     invoke():
10         rate = 30
11         show()
12
13     invoke()
14
15 scope()

```

In this `rate` example, all three `rate` cells are alive at `print`. Static lookup can select `scope.rate` but can't select sibling `invoke.rate`. Dynamic lookup can select `invoke.rate` because `invoke` is the active caller.

Declaration	Alive?	In static environment?	In dynamic environment?
<code>global rate</code>	yes	no, hidden by <code>scope.rate</code>	no, hidden by <code>invoke.rate</code>
<code>scope.rate</code>	yes	yes, first match	no, hidden by <code>invoke.rate</code>
<code>invoke.rate</code>	yes	no, off the static path	yes, first match

Treating every live cell as visible under both rules confuses lifetime with scope. Lifetime answers whether storage exists. Scope answers whether an identifier can select it.

Checkpoint 2: Account for Every Declaration (4 Points)

Use this running program:

```

1  x = 1
2
3  outer():
4      x = 2
5
6      target():
7          y = 4
8          print(x + y)
9
10     caller():
11         x = 3
12         target()
13
14     caller()
15
16 outer()

```

At the `print(x + y)` point in the running program, the live declarations are global `x = 1`, `outer's x = 2`, `caller's x = 3`, and `target's y = 4`.

1. (Construction: 2 points) Make a table with one row for each `x`. At the `print` point, record whether its cell is alive and whether it enters the static and dynamic referencing environments.

2. (Explanation: 2 points) Explain one row where lifetime and visibility have different answers.

Move a Definition, Rebuild the Chains

Worked Example for Checkpoint 3

Suggested time: 16 minutes, including the worked example.

Trace one source edit. This is the original program:

```
1  rate = 10
2
3  scope():
4      rate = 20
5
6      show():
7          print(rate)
8
9      invoke():
10         rate = 30
11         show()
12
13     invoke()
14
15 scope()
```

Now move the complete definition of `show` to global level. Keep every declaration, value, and call the same:

```
1  rate = 10
2
3  show():
4      print(rate)
5
6  scope():
7      rate = 20
8
9      invoke():
10         rate = 30
11         show()
12
13     invoke()
14
15 scope()
```

Compare the relationships first. Calculate the results after that. The move changes the textual parent of `show` from `scope` to `global`. The active caller at `print` is still `invoke` because the call statements didn't move.

Follow the change in execution order:

1. Build the new static path from the new source nesting: `show -> global`.
2. Copy the unchanged dynamic path from the active calls: `show -> invoke -> scope -> global`.
3. Stop at the first `rate` on each path. Static lookup selects `global rate = 10`. Dynamic lookup selects `invoke.rate = 30`.
4. Record only the selected binding in each referencing environment.

Moving a definition changes one lookup path

Moving `show` changes its textual parent while the active caller stays `invoke`.

Record both relationships before the move

Before the move, `scope` is the textual parent of `show`, while `invoke` is its active caller.

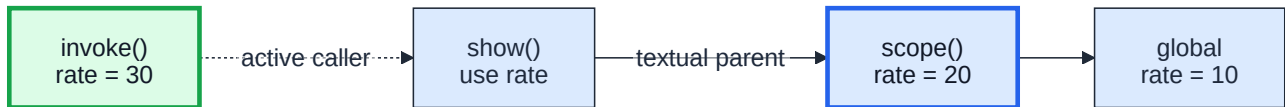


Figure 3: Record both relationships before the move

Move only the definition

After the move, `global` is the textual parent of `show`. The call from `invoke` stays in place.



Figure 4: Move only the definition

Resolve the two paths again

Static lookup now selects `global rate = 10`. Dynamic lookup still selects `invoke.rate = 30`.

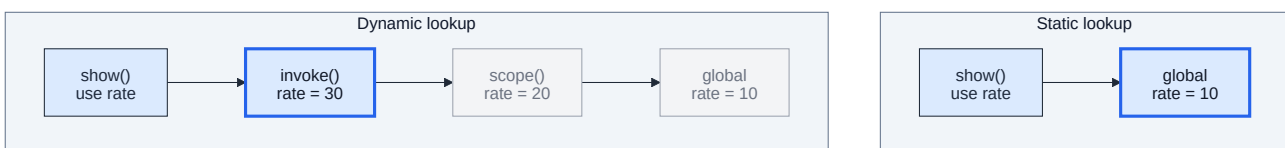


Figure 5: Resolve the two paths again

Moving a definition changes source nesting. It doesn't rewrite the call history.

Checkpoint 3: Resolve the Moved Name (4 Points)

Compare these complete programs. Program A is the original running case:

```

1  x = 1
2
3  outer():
4      x = 2
5
6      target():
7          y = 4
8          print(x + y)
9
10     caller():
11         x = 3
12         target()
13
14     caller()
15
16 outer()
```

Program B moves only the definition of `target` to global level:

```
1  x = 1
2
3  target():
4      y = 4
5      print(x + y)
6
7  outer():
8      x = 2
9
10     caller():
11         x = 3
12         target()
13
14     caller()
15
16 outer()
```

Build your trace in this order:

1. Record the changed textual parent of `target`: `outer` in Program A, then `global` in Program B.
2. Record the unchanged active call chain: `outer` -> `caller` -> `target`.
3. Build Program B's complete static and dynamic lookup paths.
4. Resolve local `y` first. Then stop at the first `x` on each path.
5. Write each referencing environment with only the selected `x` and `y` bindings.
6. Compute both results and compare them with Program A.

1. (Construction: 2 points) Draw Program B's two complete lookup paths. Mark each first `x`, write both referencing environments, and compute both results.

2. (Explanation: 2 points) Answer both parts.

- a. State which of the two printed values changes and which stays the same.
- b. Moving the definition changed one relationship and left the other intact. Identify both, and state which lookup rule depends on each.

Submit both paths, first matches, and referencing environments. A result alone doesn't identify the scope rule.